Final Document

1. Abstract

As technology becomes deeply intertwined with modern society, the ability of automated systems to distinguish human faces from everyday objects has emerged as a fundamental necessity in computer vision. Demystifying these systems reveals that they are processes grounded in rigorous mathematical foundations. The objective of this study is to clarify the mathematical framework behind facial detection and to implement a functional classification model in Python. Using Singular Value Decomposition (SVD), this project processes a dataset of XXX facial images to construct a low-dimensional principal subspace. By calculating the reconstruction error using the Euclidean norm, the model establishes a mathematical metric to effectively differentiate faces from non-facial items. Ultimately, this approach demonstrates how multivariable optimization and linear algebra can efficiently solve high-dimensional data classification challenges, bridging the gap between theoretical mathematics and practical computational applications.

2. Introduction

Poner alguna parte del planteamiento del problema acaaa*******

How do you unlock your smartphone every morning? How do security cameras in a smart circuit distinguish a human being from a moving shadow or an object? How do social media filters overlay graphics onto your face? The answer to all these everyday questions lies in the field of computer vision, specifically within facial detection and recognition systems. As you may identify, facial recognition is a tool that is everywhere.

To truly comprehend the engineering behind these systems, it is necessary to first acknowledge the massive computational challenges they face. The main challenge lies in the data overload. A low quality image of just 100 x 100 pixels has 10,000 dimensions. Processing and comparing every new object against a vast library of non-face objects is computationally expensive and inefficient. To put this into perspective, a standard High Definition (HD) image consists of 1920 x1080 pixels, which translates to over two million variables. Processing, storing, and comparing every new video frame or object against a vast library of raw pixel data in real-time is computationally prohibitive, expensive, and highly inefficient.

To overcome this geometric bottleneck, mathematical frameworks must find a way to simplify this massive amount of data without losing the crucial information that defines a human face. This is where Singular Value Decomposition (SVD) becomes an indispensable tool. SVD allows us to break down high-dimensional image matrices into their most fundamental geometric components. By focusing on these core structures, we can reduce the dimensionality of the problem, filtering out noise and redundancy.

3. Literature Review

JUAN ya lo tiene

4. Methodology

Unlike humans, computers do not perceive images visually. Instead, a digital image is mathematically represented as a matrix of numerical values. In color images, each pixel is described using the RGB (Red, Green, and Blue) model, meaning that the image is composed of three separate matrices, one for each color in the scale. Although this representation preserves detailed color information, it also significantly increases the dimensionality of the data. To simplify the computational process, the images used in this project are converted to grayscale, reducing the three-channel representation into a single matrix while still preserving the essential structural features of the face. In grayscale images, each pixel corresponds to an intensity value ranging from 0 to 255, where 0 represents black and 255 represents white.

Even after converting the dataset to grayscale, the computational complexity remains significantly elevated if the images retain their original dimensions. To mitigate this, the spatial resolution of each image is standardized to 100 x100 pixels. This reduction is achieved by resampling the original pixel density over a smaller grid while preserving the overall geometric structure and essential facial features. Throughout this resizing process, the aspect ratio is strictly maintained to prevent artificial distortions in facial proportions. As a result, each processed image retains its critical visual information while yielding a highly efficient matrix representation.

With the uniform dimensions established, the construction of the training data matrix A begins through a process known as vectorization. This technique consists of taking all the columns of the image matrix and stacking them on top of each other to transform the two-dimensional image into a single, long column vector. This mathematical transformation maps the image from a matrix space of $R^{100x100}$ into $R^{10,000}$. In result, each column of A represents a single flattened face.

Once matrix A is constructed we continue with the SVD process. SVD states that any real matrix A can be factorized into the product of three distinct constituent matrices:

$$A = U \sum V^T$$

Although the Python code does this automatically by the usage of libraries (we don't have to write all the mathematical formulas into code) it is essential to clarify the geometric and algebraic role of each matrix involved. So we will explain each building process.

The first component, U, is an orthogonal matrix (its vectors are orthogonal and normalized) composed of left singular vectors. In the context of our face detection project, each column of U retains the same spatial dimension (10,000 elements) as our flattened input images. When these columns are reshaped back into a 100 x 100 grid, they reveal abstract, ghost-like facial features known as Eigenfaces. These orthonormal columns serve as the fundamental coordinate system or base axes for our face detection framework. The earliest columns capture the most prominent geometric variations across the training set (such as overall face shape or dominant lighting gradients), providing the mathematical foundation needed to construct our lower-dimensional classification subspace.

The second component of the factorization is ∑, which is a rectangular

pseudo-diagonal matrix. The diagonal entries of this matrix, denoted as ∑, contain the

singular values of A, arranged in descending order. In our model, these singular values represent the weight or relative importance of each corresponding geometric feature in U. The larger the singular value, the more statistical variance and critical facial information that specific Eigenface captures. By analyzing the decay of these values, we can determine the optimal truncation point k, allowing the Python code to discard columns associated with lower singular values, which represent computational noise or irrelevant background details.

Finally, the matrix V^T represents the transpose of the right singular vectors. This is an orthogonal matrix whose rows provide the specific coefficients or coordinates required to reconstruct each original training image as a linear combination of the orthonormal columns in U. While V^T is essential for the internal mathematical consistency of the SVD and the initial training phase, our detection system will primarily rely on the truncated basis of U to analyze and classify completely new, unseen objects.

Immediately after the SVD algorithm computes the complete matrices, a deliberate "cutting" process known as truncation is executed in the Python code. Since the singular

values in ∑ prove that the trailing columns of U contain only noise and redundant data, the model discards them. By retaining only the first k columns of U (where <<10,000), we transform the massive U matrix into a streamlined, optimal framework denoted as the truncated matrix U. This specific matrix functions as our principal component subspace.

With the truncated facial subspace U fully isolated in the system's memory, the model is now prepared to evaluate any completely new image vector.

5. Conclusión

6. Bibliography